INSTITUTO KHIPU

Ciclo VI

Sesión 06

CARRERA PROFESIONAL

Desarrollo de Sistemas de Información

UNIDAD DIDÁCTICA

Despliegue y Operación DEVOPS

Tema: Fundamentos de CI/CD - Conceptos y Herramientas.

Sesión 06: Fundamentos de CI/CD: Conceptos y Herramientas.

Introducción.

Vamos a trabajar en una actividad práctica sobre **CI/CD en DevOps**. La idea es que veas cómo se implementa un flujo de integración y entrega continua usando herramientas clave.

🚀 Flujo típico de CI/CD en DevOps

1. **Código fuente**
   - Se gestiona en repositorios como **GitHub**, **GitLab** o **Bitbucket**.
2. **Integración continua (CI)**
   - Cada vez que un desarrollador hace un *commit*, se ejecutan pruebas automáticas.
   - Herramientas: **Jenkins**, **GitHub Actions**, **GitLab CI/CD**, **Azure DevOps Pipelines**, **CircleCI**.
3. **Entrega continua (CD)**
   - Si las pruebas pasan, el sistema despliega automáticamente en un entorno de pruebas o producción.
   - Herramientas: **ArgoCD**, **Spinnaker**, **FluxCD**, **Azure DevOps**, **AWS CodePipeline**.
4. **Monitoreo y retroalimentación**
   - Se usan sistemas como **Prometheus**, **Grafana**, **ELK Stack** o **Datadog** para verificar que el despliegue funciona correctamente.

🛠 Herramientas destacadas

Etapa	Herramienta	Uso principal
Repositorio	GitHub / GitLab	Control de versiones
CI	Jenkins / GitHub Actions	Compilación y pruebas automáticas
CD	ArgoCD / Spinnaker	Despliegue automatizado
Contenedores	Docker / Kubernetes	Empaquetado y orquestación
Infraestructura	Terraform / Ansible	Infraestructura como código
Monitoreo	Prometheus / Grafana	Observabilidad y métricas

💡 Actividad práctica propuesta

1. **Configura un repositorio en GitHub** con un proyecto sencillo (ejemplo: aplicación web en Node.js o Python).
2. **Agrega un pipeline de CI/CD** usando GitHub Actions:
   - Paso 1: Ejecutar pruebas unitarias.
   - Paso 2: Construir una imagen Docker.
   - Paso 3: Desplegar en un clúster de Kubernetes (puede ser Minikube o AKS/EKS/GKE).
3. **Monitorea el despliegue** con Prometheus y Grafana para validar que la aplicación está corriendo correctamente.

En base a lo anterior, desarrollaremos la siguiente actividad.

Vamos a trabajar con **GitHub Actions** desde cero, cubriendo instalación, configuración y uso antes de pasar al ejercicio principal de CI/CD.

🔧 1. Instalación y preparación

GitHub Actions no requiere instalación en tu máquina, ya que está integrado directamente en GitHub. Lo que sí necesitas es:

- **Cuenta en GitHub** (gratuita o de pago).
- **Repositorio** donde alojar tu proyecto.
- Opcional: **Git instalado en tu PC** para clonar y subir cambios.

👉 Pasos iniciales:

1. Crea una cuenta en GitHub.
2. Crea un nuevo repositorio (ejemplo: `ci-cd-demo`).
3. Clona el repositorio en tu máquina:

   bash

   ```
   git clone https://github.com/tuusuario/ci-cd-demo.git
   cd ci-cd-demo
   ```

⚙ 2. Configuración de GitHub Actions

GitHub Actions funciona con **workflows** definidos en archivos YAML dentro de la carpeta `.github/workflows`.

1. En tu repositorio, crea la carpeta:

```bash
mkdir -p .github/workflows
```

2. Dentro, crea un archivo `ci.yml` (puede tener cualquier nombre).
3. Estructura básica de un workflow:

```yaml
name: CI Pipeline

on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v3

      - name: Set up Node.js
        uses: actions/setup-node@v3
        with:
          node-version: '18'

      - name: Install dependencies
        run: npm install

      - name: Run tests
        run: npm test
```

▶ 3. Uso básico

- Cada vez que hagas un **push** a la rama `main`, GitHub ejecutará este workflow.
- Puedes ver el progreso en la pestaña **Actions** de tu repositorio.
- Si las pruebas pasan, verás un ✅ verde; si fallan, un ❌ rojo con detalles del error.

Adaptamos la práctica a un **CRUD básico en Python con interfaz gráfica**. Esto les permitirá trabajar con lo que ya conocen y, al mismo tiempo, integrar la lógica de CI/CD con GitHub Actions.

🛠 1. Preparación del entorno

- Instalar **Python 3.x**.
- Instalar librerías necesarias:

  bash

  ```
  pip install tkinter
  pip install pytest
  ```

  (Tkinter suele venir incluido en la instalación estándar de Python, pero conviene verificar.)

📂 2. Estructura del proyecto

Código

```
crud-python-gui/
│
├── app.py              # Aplicación principal con Tkinter
├── crud.py             # Lógica CRUD (crear, leer, actualizar,
eliminar)
├── tests/
│   └── test_crud.py    # Pruebas unitarias con pytest
└── requirements.txt    # Dependencias
```

📄 3. Ejemplo de CRUD con Tkinter

crud.py (lógica básica con lista en memoria):

python

```python
data = []

def create(item):
    data.append(item)
    return True

def read():
    return data

def update(index, new_item):
    if 0 <= index < len(data):
        data[index] = new_item
        return True
    return False
```

```
def delete(index):
    if 0 <= index < len(data):
        data.pop(index)
        return True
    return False
```

app.py (interfaz gráfica con Tkinter):

python
```
import tkinter as tk
import crud

def add_item():
    item = entry.get()
    if item:
        crud.create(item)
        refresh_list()

def refresh_list():
    listbox.delete(0, tk.END)
    for i, item in enumerate(crud.read()):
        listbox.insert(tk.END, f"{i}: {item}")

root = tk.Tk()
root.title("CRUD Básico en Python")

entry = tk.Entry(root)
entry.pack()

add_button = tk.Button(root, text="Agregar", command=add_item)
add_button.pack()

listbox = tk.Listbox(root)
listbox.pack()

refresh_list()
root.mainloop()
```

□ 4. Pruebas unitarias con pytest

tests/test_crud.py:

python
```
import crud

def test_create_and_read():
    crud.create("item1")
    assert "item1" in crud.read()

def test_update():
    crud.create("item2")
    assert crud.update(0, "item2_updated")
    assert crud.read()[0] == "item2_updated"

def test_delete():
    crud.create("item3")
```

```
    assert crud.delete(0)
    assert "item3" not in crud.read()
```

⚙ 5. Configuración de GitHub Actions

En `.github/workflows/ci.yml`:

yaml
```yaml
name: Python CI

on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v3

      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.10'

      - name: Install dependencies
        run: pip install -r requirements.txt

      - name: Run tests
        run: pytest
```

▶ 6. Flujo práctico

1. Los alumnos crean el CRUD en Python con Tkinter.
2. Añaden pruebas unitarias con pytest.
3. Configuran el pipeline en GitHub Actions.
4. Cada vez que hagan un **push**, se ejecutarán automáticamente las pruebas.

🚀 1. Objetivo del CD

El propósito es que, una vez que el código pase las pruebas en CI, se **empaquete automáticamente** y quede listo para distribución o despliegue. En aplicaciones gráficas de escritorio en Python, las opciones más comunes son:

- **PyInstaller** → genera ejecutables (.exe, .app, etc.).
- **Docker** → empaqueta la aplicación en un contenedor (más útil si se quiere distribuir en entornos controlados).

Para este caso, usaremos **PyInstaller**, ya que es lo más práctico para aplicaciones gráficas.

🛠 2. Configuración de empaquetado con PyInstaller

1. Instalar PyInstaller:

   bash

   ```
   pip install pyinstaller
   ```

2. Crear ejecutable:

   bash

   ```
   pyinstaller --onefile app.py
   ```

   Esto generará un archivo ejecutable en la carpeta `dist/`.

3. Añadir a `requirements.txt`:

   Código

   ```
   pytest
   pyinstaller
   tkinter
   ```

⚙ 3. Workflow de GitHub Actions para CD

Ampliamos el pipeline (`.github/workflows/ci-cd.yml`) para incluir empaquetado:

yaml

```yaml
name: Python CI/CD

on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v3

      - name: Set up Python
        uses: actions/setup-python@v4
        with:
```

```
        python-version: '3.10'

    - name: Install dependencies
      run: pip install -r requirements.txt

    - name: Run tests
      run: pytest

    - name: Build executable
      run: pyinstaller --onefile app.py

    - name: Upload artifact
      uses: actions/upload-artifact@v3
      with:
        name: python-crud-executable
        path: dist/app
```

🎁 4. Resultado

- Cada vez que se haga un **push a main**, se ejecutarán las pruebas (CI).
- Si todo pasa, se generará un **ejecutable** y se subirá como **artifact** en GitHub Actions.
- Los alumnos podrán descargar el ejecutable directamente desde la pestaña **Actions → Artifacts**.

🔍 5. Extensiones posibles

- **Distribución automática**: subir el ejecutable a GitHub Releases.
- **Dockerización**: empaquetar la app en un contenedor para despliegue en servidores.
- **Notificaciones**: integrar con Slack/Teams para avisar cuando el pipeline termina.

☞ Con esto ya tienen un **ciclo CI/CD completo para un CRUD en Python con GUI**: pruebas automáticas y empaquetado listo para distribución.